eBPF Japan Meetup #8

eBPF でカーネルの転送情報の変更をどこまで調停できるか

IP 経路・IP neighbor・FDB エントリの変更を観測して止めよう！

Kazuya Goda

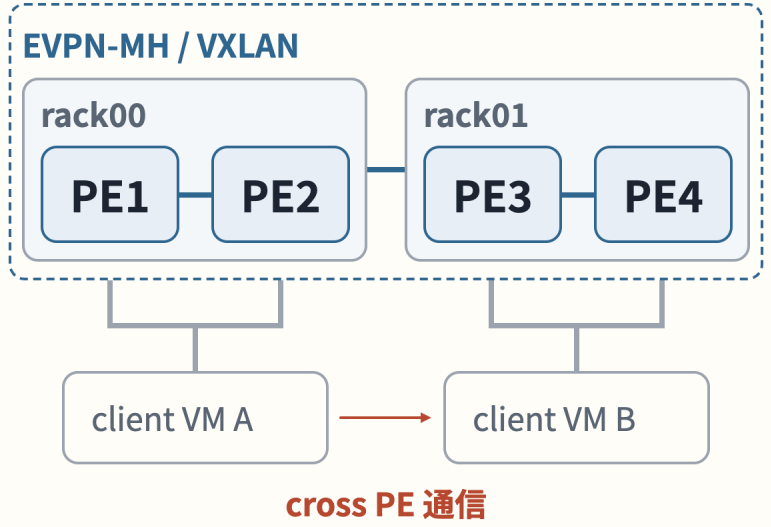
EVPN-MH 検証中に FRR を再起動したら復旧後に数十秒経ってから通信断が発生

環境

- FRR による EVPN/VXLAN を Multihoming 構成で利用
- Bridge FDB エントリと IP neighbor を静的に登録
- クライアントの MAC と IP は事前に分かっているので動的学習なし

症状

- FRR を systemctl restart frr.service で再起動
- 配下のクライアント間の通信が ARP 解決ができない状態となる



原因: zebra が static に登録したエントリを上書き

1. **FRR を起動**
2. **static に FDB エントリ、IP neighbor を登録**
3. **FRR を停止**
   - ここで IP neighbor が書き換わり、ARP の state が PERMANENT から REACHABLE に変化
4. **FRR を起動**
   - REACHABLE のまま 60 秒後に STALE、FAILED と遷移
     - 動的学習はしないようになっていたので ARP に応答せず FAILED となってしまっていた

ここでのポイントは転送情報が意図せず上書きされていたこと

FRR の EVPN-MH の問題については ENOG91 での資料で (こちらも eBPF 使ってます)
https://enog.jp/wordpress/wp-content/uploads/2026/09/enog91_frr_evpn_mh.pdf

転送情報の変更を調停する仕組みがないのが問題なのでは

1. IP 経路や IP neighbor には「誰のものか」のタグがある（proto）
2. しかし **CAP_NET_ADMIN** を持っていれば誰でもエントリを置換・削除できる
3. 複数のデーモンが同じ転送情報を触ると、意図しない変更が起きる
   例: systemd-networkd が zebra の nexthop を消す issue（systemd #29034）

パケット転送情報の正常性を保ちたい！

転送情報 = IP 経路、IP neighbor、bridge FDB のエントリ

正常性を保つためにやること

観測

- 意図しない変更に気づく
- 誰が、どの経路で変えたか

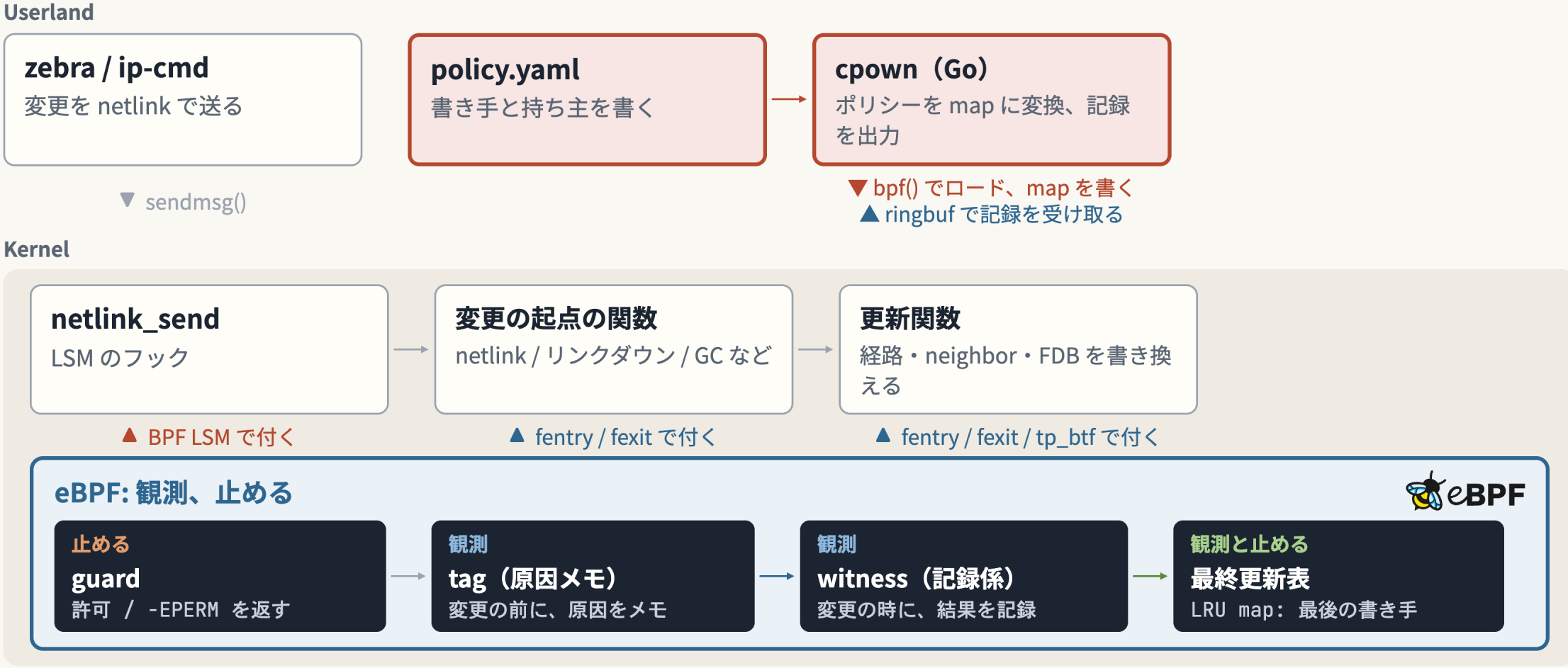
止める

- 持ち主以外の変更を拒否

戻す

- 止められなかった変更を戻す
- 今回は対象外

cpown: 転送情報の変更を調停するシステム全体像



- **観測**: tag が原因のメモを置き、witness が全部の変更を、原因と書き手つきで記録
- **止める**: guard が、netlink の要求を判定して拒否。更新表は、観測が書き、guard が読む

観測

誰がどうやって変更したかを記録する

観測

観測で知りたい４つのこと

誰が

変更したのは誰か
例: cgroup やプロセス名

何を

どのエントリが、どう変わったか
例: proto が static → zebra

なぜ

userland の要求か、カーネル内部から呼び出されたか
例: リンクダウン、IP neighbor の GC、FDB エージング

前の書き手と異なるか

違えば conflict として出す
※ 経路は、claim した IPv4 の prefix だけ

観測

3 つのコンポーネントで実現

tag（原因のタグ付け）

変更の起点になる関数に事前に原因の種別をタグ付けしておく

答える問い

なぜ

witness（記録係）

経路・neighbor・FDB の変更処理関数に付ける。変更時に結果を記録する。

答える問い

何が・誰が

最終更新表

エントリごとに、最後の書き手を覚える。次の書き込みと比べる。

答える問い

前の書き手と違うか

hook の種類（fentry / fexit / tp_btf）と、付ける関数の一覧は、このあと。hook の種類は付録

観測

cpown: 記録は kernel 中の eBPF で取る

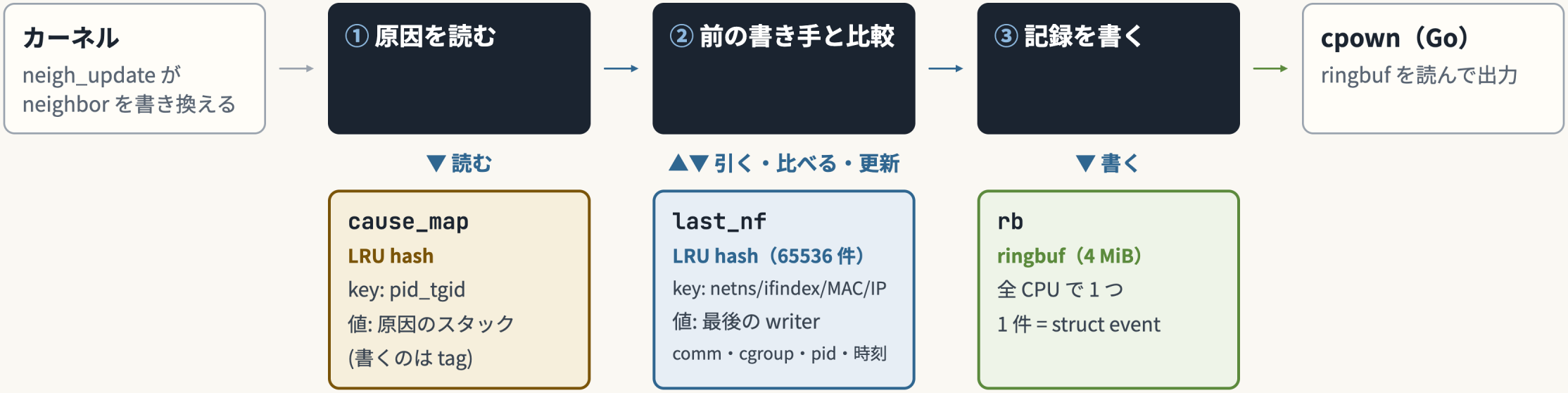
tag で原因を決め witness が結果を記録してメモを添える



観測

witness で登場する eBPF map と用途

例: neigh_update が呼ばれたとき（tp_btf/neigh_update の witness）



- last_nf は「今の状態」を持っていてエントリごとに最後の書き手を 1 件保存
- rb は「過去の記録」を持っていて変更のたびに 1 件ずつ入る

観測

どのタイミングで、誰が、何を読み書きするか

タイミング	カーネル	eBPF プログラム	map	cpown（Go）
起動時 **cpown が準備**			watch4 に監視する prefix を**書く** （IPv4 経路のみ）	**bpf() でロード・attach** policy.yaml を読む
① **起点に入る** 変更の前	`fib_netdev_event` に入る 例: リンクダウン	**tag（fentry）** `cause_push(LINK)`	**`cause_map`** 原因を積んで**書く**	

②
更新関数が書き換える
変更の時
	`fib_notify_alias_delete`
	経路が消える
		witness
		`cause_cur()`
		`last_nf / owner4` を引いて比べる
		`event` を `submit`
			cause_map: **読む**
			last_nf・owner4: **読み書く**
			rb: **書く**	

③ **起点を出る** 変更の後	`fib_netdev_event` を出る	**tag（fexit）** `cause_pop()`	**`cause_map`** 原因を降ろす（消す）	
いつでも **別の流れ** 非同期			rb: **読む**	**ringbuf を読み、名前を引いて出力** `-witness`

読む: map から取り出す　**書く**: map に入れる。 tag は原因を積むだけ。witness が、原因を読み、前の書き手と比べ、記録を出す。

観測

hook する関数

結果を記録（witness）

対象・操作	hook
IPv4 経路 追加	fentry/rtmsg_fib
IPv4 経路 削除	fentry/fib_notify_alias_delete
IPv6 経路 追加	fexit/fib6_add
IPv6 経路 削除	fexit/fib6_del
neighbor 更新	tp_btf/neigh_update
neighbor 削除	tp_btf/neigh_cleanup_and_release
FDB 追加	tp_btf/br_fdb_add
FDB 削除	tp_btf/fdb_delete
FDB 学習・更新	tp_btf/br_fdb_update

原因を書く（tag） fentry + fexit

原因	関数
NETLINK	rtnetlink_rcv_msg
LINK	fib_netdev_event addrconf_notify rt6_disable_ip neigh_ifdown neigh_carrier_down
ADDR	fib_inetaddr_event inet6_addr_add inet6_rtm_deladdr addrconf_del_ifaddr
NEXTHOP	remove_nexthop nexthop_flush_dev
NEIGH_GC	neigh_periodic_work
FDB_AGE	br_fdb_cleanup
FDB_PORT	br_fdb_delete_by_port
RA	ndisc_router_discovery

- 使い分け: tracepoint があれば tp_btf、なければ fentry / fexit

観測

(参考) witness と tag のコード抜粋

tag: 起点の入口で積み、出口で降ろす

```
#define TAG(fn, c)
  SEC("fentry/" #fn)
  int BPF_PROG(in_##fn)
  { cause_push(c); return 0; }
  SEC("fexit/" #fn)
  int BPF_PROG(out_##fn)
  { cause_pop(); return 0; }

TAG(rtnetlink_rcv_msg, C_NETLINK)
TAG(fib_netdev_event, C_LINK)
TAG(neigh_periodic_work, C_NEIGH_GC)
TAG(br_fdb_cleanup, C_FDB_AGE)
// ... 全部で 16 個
```

witness: 更新関数で、原因を読み、比べて、記録する

```
SEC("tp_btf/neigh_update")
int BPF_PROG(t_neigh_update,
             struct neighbour *n,
             const u8 *lladdr, u8 new,
             u32 flags, u32 nlmsg_pid)
{
  struct event *e = ev_new(ctx,
    K_NEIGH, OP_UPD,
    nlmsg_pid ? C_NETLINK : cause_cur());
  ...
  if (e→cause == C_NETLINK) {
    nf_key_neigh(&k, e);
    nf_check(e, &k, false);
  }
  bpf_ringbuf_submit(e, 0);
  return 0;
}
```

bpf/observe.bpf.c から抜粋（整形・一部省略）

観測

観測例 1: 誰が書き換えたか観測

policy.yaml

```
owners:
  operator: { cgroup: /operator }
  frr:      { cgroup: /frr }
claims:
  - name: br100-fdb
    fdb: { bridge: br100, mac: "02:00:00:00:00:20" }
    owner: operator
    mode: audit
```

何が
PERMANENT → NOARP　proto は zebra に

誰が・なぜ
zebra（/frr）　netlink の要求

前の書き手
ip（/operator）　conflict として出る

```
$ cpown -policy policy.yaml -witness
[operator] # ip neigh replace 192.168.100.20 lladdr 02:00:00:00:00:20 dev br100 nud permanent
NEIGH UPD 192.168.100.20 02:00:00:00:00:20 NONE->PERMANENT proto=static
     cause=NETLINK by ip/ip[421] cg=/operator
[root] #  (FRR を起動)
NEIGH UPD 192.168.100.20 02:00:00:00:00:20 PERMANENT->NOARP proto=zebra
     cause=NETLINK by zebra/zebra_dplane[451] cg=/frr
  !! CONFLICT[writer] previous owner: ip[421] cg=/operator state=PERMANENT
```

観測

観測例 2: なぜ経路が書き換えられたか観測

policy.yaml

```
owners:
  operator: { cgroup: /operator }
claims:
  - name: mgmt-routes
    route: { prefix: 10.9.0.0/16, scope: subtree }
    owner: operator
    mode: audit
```

何が
経路 10.9.0.0/24 の削除　proto は static

なぜ
cause=LINK ip route del ではなく、リンクダウンの連鎖

誰が
ip[382]　リンクダウンを実行したプロセス

```
[operator] # ip route add 10.9.0.0/24 via 10.1.0.2 proto static
$ cpown -policy policy.yaml -witness
[operator] # ip link set d0 down
ROUTE4 DEL 10.9.0.0/24 via 10.1.0.2 dev=d0 proto=static cause=LINK by ip/ip[382] cg=/operator
ROUTE4 DEL 10.1.0.0/24 dev=d0 proto=kernel cause=LINK by ip/ip[382] cg=/operator
ROUTE4 DEL 10.1.0.255/32 dev=d0 proto=kernel cause=LINK by ip/ip[382] cg=/operator
```

観測

eBPF の利点: 変更した瞬間のスタックトレースが取れる

ほかの観測との違い

ip monitor などは、変わった結果しか見えない
書き換えた瞬間の、コードの経路は見えない

カーネルのスタック

変更に至る関数の連鎖
bpf_get_stackid

ユーザのスタック

呼び出したプログラムの関数と行
BUILD_ID と gdb でシンボル化

```
NEWROUTE proto=200 dst=192.0.2.0/24 exe=ip

  [k] bpf_lsm_netlink_send+0x5
  [k] netlink_sendmsg+0x1ea
  [k] ____sys_sendmsg
  [k] do_syscall_64

  __libc_sendmsg sendmsg.c:28
  iproute_modify iproute.c:1597
  do_cmd ip.c:143
  main ip.c:327
```

出力は PoC（tools/observe -s）。cpown 本体は、スタックを取らない

PART 2

止める

止める

全体像: 転送情報を変更するかの判定を eBPF でやる

Userland

zebra / ip-cmd
変更を netlink で送る

▼ sendmsg()

policy.yaml
書き手と持ち主を書く

→

cpown（Go）
ポリシーを map に変換

▼ bpf() でロード、map を書く

Kernel

netlink ソケット
send() を受ける

→

netlink_send
LSM のフック

→

BPF LSM eBPF

```
guard Prog:
  lsm/netlink_send
```

```
BPF map:
owners / claims
```

→

rtnetlink
経路 / neighbor / FDB を更新

▼ 違反なら -EPERM
send() が失敗し、変更は入らない

違反なし → 0（許可）

① policy.yaml と cpown
「これは誰のものか」を記述し BPF map に変換

② BPF map
書き手と claim（持ち主）を入れておく

③ guard（eBPF Prog）
netlink_send で 許可/拒否(-EPERM) を返す

止める

変更内容が見えて拒否もできるのはどこか

両方を満たせるのは BPF LSM の netlink_send

候補	要求の中身	拒否
kprobe / fentry	見える	できない（戻り値を変えられない）
seccomp	引数まで（iov の中身は見えない）	できる
BPF LSM netlink_send	見える（コピー後の skb）	できる（-errno）

止める

LSM: Linux Security Modules

netlink で要求を送る
例: 経路の追加・削除

→

`security_netlink_send`
LSM hook

→

SELinux / AppArmor 等
どれか 1 つでも拒否なら拒否

→

0: 許可　-errno: 拒否
結果

操作	hook	判定の例
プログラムの実行	security_bprm_check	/tmp 配下のバイナリは実行させない
ファイルを開く	security_file_open	nginx は /etc/shadow を開けない
接続する	security_socket_connect	DB サーバは外部へ接続させない
シグナルを送る	security_task_kill	他のユーザのプロセスは kill できない
他プロセスを覗く	security_ptrace_access_check	親子以外への ptrace を禁止する
netlink を送る	security_netlink_send	今回の話

止める

BPF LSM: LSM の Hook に BPF を付ける

```
SEC("lsm/netlink_send")
int BPF_PROG(guard, struct sock *sk, struct sk_buff *skb)
{
  if (BPF_CORE_READ(sk, sk_protocol) =/= NETLINK_ROUTE)
    return 0;
  ...
  return mc.ret ? -EPERM : 0;
}
```

項目	中身
BPF LSM	LSM の hook に BPF プログラムを付ける（v5.7 以降）
戻り値	0 で許可、-errno で拒否
他の LSM	拒否を足すだけ（他の拒否は覆せない）
前提	CONFIG_BPF_LSM=y、起動パラメータ lsm=…,bpf

止める

kernel 側の LSM Hook Point

① net/netlink/af_netlink.c　netlink_sendmsg()

```
memcpy_from_msg(skb_put(skb, len), msg, len);  /* 要求を skb にコピー */
err = security_netlink_send(sk, skb);          /* ← hook */
if (err) {
  kfree_skb(skb);
  goto out;  /* send() が err で失敗する。要求は届かない */
}
err = netlink_unicast(sk, skb, dst_portid, ...);  /* 許可なら rtnetlink へ */
```

② security/security.c　LSM の入口

```
int security_netlink_send(
  struct sock *sk, struct sk_buff *skb)
{
  return call_int_hook(
    netlink_send, sk, skb);
}
```

③ security/bpf/hooks.c　起動時に登録

```
static struct security_hook_list
  bpf_lsm_hooks[] = {
  LSM_HOOK_INIT(netlink_send,
    bpf_lsm_netlink_send),
  ...
};
```

④ kernel/bpf/bpf_lsm.c　BPF を付ける場所

```
noinline int bpf_lsm_netlink_send(
  struct sock *sk,
  struct sk_buff *skb)
{
  return 0;  /* 空の関数 */
}
```

- 要求は skb にコピーされたあとで、hook に渡る（中身が見える）
- ③ で、BPF LSM が自分の実装として登録しているので、② の呼び出しが ④ に届く
- ④ に付けた BPF プログラムが 0 以外を返すと、残りの LSM は呼ばれず、send() がその値で失敗する

止める

guard の処理の流れ



拒否したら -EPERM　send() 全体が失敗し、何も変わらない

この位置の性質	意味
TOCTOU がない	検査する skb と、実行される skb が同じ
送った本人の文脈	cgroup とプロセス名が、そのまま取れる
ユーザ空間の要求だけ	カーネルが作る netlink は通らない

止める

guard でやること

1. 対象を絞る

NETLINK_ROUTE 以外は許可

2. 書き手を決める

cgroup（祖先を最大 8 階層）とプロセス名

3. 全メッセージを見る

bpf_loop で最大 4096 メッセージ、各 64 属性

4. claim を引く

IPv4 の経路（exact → LPM）・neighbor、FDB

5. 判定する

持ち主でも allow でもなければ違反

6. 返す

違反が 1 つでもあれば send() 全体を -EPERM
解析できない要求も、enforce の claim があれば拒否

止める

(参考) guard のコード抜粋

書き手を決めて、違反かを判定する

```
__u8 actor_owner(const char *comm)
{
  ...
  for (int i = 0; i < MAX_CG_DEPTH; i++) {
    k.cgid = bpf_get_current_ancestor_cgroup_id(level - i);
    o = bpf_map_lookup_elem(&cg_owner, &k);
    if (o)
      return *o;    // 近い祖先の持ち主
  }
  return 0;
}

bool violates(const struct claim *c, __u8 actor, __u8 op)
{
  if (actor == c->owner)
    return false;
  if ((c->allow[actor] & op) == op)
    return false;
  return true;
}
```

netlink_send で全メッセージを見て、返す

```
SEC("lsm/netlink_send")
int BPF_PROG(guard, struct sock *sk,
             struct sk_buff *skb)
{
  if (BPF_CORE_READ(sk, sk_protocol)
      != NETLINK_ROUTE)
    return 0;
  mc.gc.actor = actor_owner(comm);
  ...
  bpf_loop(MAX_MSGS, msg_cb, &mc, 0);
  // msg_cb: do_route / do_neigh /
  // do_fdb が claim を引き、
  // violates() なら mc->ret = -EPERM

  // 1 つでも違反なら send 全体
  return mc.ret ? -EPERM : 0;
}
```

bpf/guard.bpf.c から抜粋（整形・一部省略）

policy.yaml: 「誰のものか」「誰が操作できるか」を書く

```
owners:
  operator: { cgroup: /operator }
  frr:      { cgroup: /frr }
claims:
  - name: mgmt-routes
    route: { prefix: 10.9.0.0/16 }
    owner: operator
    allow: { frr: [add] }
    mode: enforce
```

キー	中身
owners	書き手に名前を付ける（cgroup とプロセス名）
claims	対象（IPv4 経路・neighbor・FDB）と、その持ち主
allow	持ち主以外に許す操作（add / replace / delete）
mode	observe・audit（止めずに記録。表示だけ違う）、enforce（拒否）

止める

どのタイミングで、誰が、何を読み書きするか

タイミング	カーネル	eBPF プログラム（guard）	map	cpown（Go）
起動時 **cpown が準備**			claims・cg_owner・gconfig: **書く** owner4: 既存の経路を**書く**	**policy.yaml を map に変換** bpf() でロード・attach
1 秒ごと **cpown が更新**			cg_owner・claims: **書く** port_master: **書く** (新しい cgroup・ifindex)	**cgroup と ifindex を引き直す**
① **send() が来る** 要求を判定	**netlink_send** 要求が skb にコピーされた後	**guard（lsm/netlink_send）** 誰か: cg_owner を引く メッセージごとに claim を引く 違反なら -EPERM	cg_owner: **読む** route_exact・route_claims: **読む** neigh_claims・fdb_claims: **読む** proto_owner・gconfig: **読む**	
② **違反のとき** 記録を出す	**send() が失敗** -EPERM で、変更は入らない	**guard** 前の書き手を引く event を submit	owner4・last_nf: **読む** grb: **書く**	
いつでも **別の流れ** 非同期			grb: **読む**	**DENIED を出力** 誰が・何を・どの claim

読む: map から取り出す　**書く**: map に入れる。　owner4・last_nf は、観測の witness が書く表（「前の書き手」を出すために読む）。

止める

実行例: FDB エントリを上書きさせない

検証の構成

R1（観測対象の PE）
VTEP: 10.0.0.1 (bridge br100 + vx100)
netdev: bridge br100 + vx100
static に 02:00:00:00:00:20 を登録

VXLAN
EVPN

R2（対向の PE）
VTEP: 10.0.0.2
netdev: bridge br100 + vx100
CE: 配下に H2 と H3

H2 claim の対象
02:00:00:00:00:20

H3 対象外
02:00:00:00:00:30

```
# R1 で運用者（cgroup /operator）が登録
bridge fdb add 02:00:00:00:00:20
  dev vx100 master static
```

policy.yaml（claim は MAC 1 つ）

```
owners:
  operator: { cgroup: /operator }
  frr:      { cgroup: /frr }
claims:
  - name: br100-fdb
    fdb: { bridge: br100, mac: "02:00:00:00:00:20" }
    owner: operator
    mode: enforce
```

止める

FDB エントリの上書きが止まる

```
[operator] # bridge fdb add 02:00:00:00:00:20 dev vx100 master static
$ cpown -policy policy.yaml
[root] #  (R2、R1 の順に FRR を起動)
0.000 DENIED FDB replace 02:00:00:00:00:20 br=br100 [violation] by zebra[446] cg=/frr claim=br100-fdb
zebra.log: ZEBRA: [TF6P2-X0GVD][EC 100663299] netlink_send_msg error: Operation not permitted
[root] # bridge fdb show br br100 | grep 02:00:00:00:00:20
02:00:00:00:00:20 dev vx100 vlan 1 master br100 static
02:00:00:00:00:20 dev vx100 master br100 static
```

(参考) audit: static エントリが extern_learn (EVPN) のエントリに置き換わる

```
02:00:00:00:00:20 dev vx100 vlan 1 extern_learn master br100 static
02:00:00:00:00:20 dev vx100 extern_learn master br100 static
02:00:00:00:00:20 dev vx100 dst 10.0.0.2 self extern_learn
```

止める（課題）

課題1: claim 対象外の MAC にも影響が...

H3 (02:00:00:00:00:30) は claim の対象外のため bridge の FDB に登録されるはず

(参考) 本来 H3 の MAC は、EVPN のエントリとして入る

```
02:00:00:00:00:30 dev vx100 vlan 1 extern_learn master br100
02:00:00:00:00:30 dev vx100 extern_learn master br100
02:00:00:00:00:30 dev vx100 dst 10.0.0.2 self extern_learn
```

しかし extern_learn も、リモート VTEP を指すエントリも入らない

```
[root] # bridge fdb show br br100 | grep 02:00:00:00:00:30
02:00:00:00:00:30 dev vx100 master br100    ← extern_learn が無い
[root] # bridge fdb show dev vx100 | grep self | grep 02:00:00:00:00:30
（出力なし: リモート VTEP を指す self のエントリが無い）
cpown の出力: DENIED FDB replace 02:00:00:00:00:20 …  ← 02:00:00:00:00:30 の拒否ログは無い
```

課題1: …:20 と …:30 は同じ sendmsg() に載っている

このため 02:00:00:00:00:20 の拒否に巻き込まれて 02:00:00:00:00:30 も登録されない




```
zebra の netlink send（BPF で数えた）
send #1  4 messages  [ …:20  …:30  …:00  …:00 ]   verdict -1 (EPERM)
send #2  1 message   [ …:00 ]                      verdict 0
send #3  1 message   [ …:20 ]                      verdict 0
send #4  1 message   [ …:30 ]                      verdict 0
```

verdict は BPF LSM のフックが返す戻り値

課題2: 現状止められるかは変更の契機で決まる

入口	例	止められるか
netlink	ip、zebra、networkd	止められる（BPF LSM で変更の前に判定）
ioctl	古い route / arp	今はすり抜ける（security_file_ioctl で止められる。未実装）
IPv6 の経路・neighbor	ip -6、zebra	今は判定しない（未実装）
カーネル内部	リンクダウン、GC、エージング、RA	止められない（拒否できる hook がない）

止める対象は、ユーザ空間のデーモンが netlink で送る要求
カーネル内部の変更は、観測で見るだけ

まとめ

まとめ

できたこと、できないこと

観測: できた

- netlink とカーネル内部の変更が、hook した関数を通る限り見える
- 原因（LINK、GC など）と書き手もわかる
- 書き換えの conflict を特定できる

止める: 難しい

- 止められるのは netlink だけ（カーネル内部は無理）
- 拒否は send() 単位で実行される
- 新規と置換を区別できない（付録の実行例 1）
- 返せるのは errno だけ

どこで調停すべきか、そもそも調停は必要なのか

- eBPF だけでは限界
- 止められない変更が残る
- 拒否は send() 単位
- 転送情報に持ち主を持たせる
- ファイルの所有者と権限のように
- カーネルが検査する
- そもそも複数で管理させない
- 1 つのデーモンに寄せる
- 運用で解くのが正当かもしれない

Appendix

観測

ringbuf と LRU map の特徴

ringbuf

eBPF → ユーザ空間 の一方向キュー

- 全 CPU で 1 本。確保した順に並ぶ（順序が崩れない）
- reserve → submit。確保した領域に直接書くので、コピーが少ない
- 満杯だと reserve が失敗し、**イベントが落ちる**
- 落ちた数は drops map に数える
- サイズは 4 MiB。cpown が読む

LRU map

キーで引く表。満杯なら古いものを捨てる

- 満杯でも update が失敗しない。使われていないものが追い出される
- eBPF から lookup / update / delete できる
- 追い出されたエントリの前の書き手は、**不明**になる
- cause_map: fexit が漏れても、溜まり続けない
- last_nf・owner4: 65536 件

map	種類	件数	書く側	中身
cause_map	LRU hash	16384	tag	task ごとの原因のスタック
last_nf	LRU hash	65536	witness	neighbor・FDB の最後の書き手
owner4	LRU hash	65536	witness	IPv4 経路の最後の書き手（監視する prefix だけ）
rb	ringbuf	4 MiB	witness	変更の記録（cpown が読む）

観測

fentry / fexit / tp_btf

種別	付く場所	使う理由
fentry	カーネル関数の入口	引数を型付きで読む（BPF trampoline）
fexit	カーネル関数の出口	戻り値も読む（成功したかが分かる）
tp_btf	tracepoint	構造体の中身（IP・MAC・状態）を直接読む。関数名より安定

```
SEC("tp_btf/fdb_delete")
int BPF_PROG(t_fdb_delete, struct net_bridge *br, struct net_bridge_fdb_entry *f)

SEC("fexit/fib6_add")
int BPF_PROG(w_fib6_add, struct fib6_node *root, struct fib6_info *rt, ..., int ret)
```

- 使い分け: tracepoint があれば tp_btf、なければ fentry / fexit
- どれも見るだけ（戻り値は変えられない）。止めるのは BPF LSM

前提: CONFIG_DEBUG_INFO_BTF、DYNAMIC_FTRACE_WITH_DIRECT_CALLS。構造体は CO-RE（BPF_CORE_READ）で読む。kprobe は使っていない

止める

proto: 経路に付く、書き手のラベル

カーネルは proto を保存するだけで、意味を決めない。
誰でも、好きな値で書ける。

proto	番号	意味	FRR 起動時の扱い
static	4	カーネル定義。手動で設定した経路	見なさない（残る）
boot	3	カーネル定義。proto を省いた ip route add の既定値	見なさない（残る）
196	196	FRR の staticd が入れる経路（RTPROT_ZSTATIC）	自分の経路と見なし、消す
186	186	FRR の BGP が入れる経路（ip route show では bgp と表示）	自分の経路と見なし、消す

止める

実行例 1（1/3）：運用者の経路 4 本と、ポリシー

```
# 運用者（cgroup /operator）
ip route add 10.9.1.0/24 via 10.1.0.2 proto static metric 20
ip route add 10.9.2.0/24 via 10.1.0.2 proto boot metric 20
ip route add 10.9.3.0/24 via 10.1.0.2 proto 196 metric 20
ip route add 10.9.4.0/24 via 10.1.0.2 proto 186 metric 20
```

policy.yaml

```
owners:
operator: { cgroup: /operator }
frr: { cgroup: /frr }
claims:
- name: mgmt-routes
route: { prefix: 10.9.0.0/16, scope:
subtree }
owner: operator
allow: { frr: [add] }
mode: enforce
```

① FRR を起動した直後

zebra が、proto 196 と 186 の経路を「前回の残骸」と見て、消す（10.9.3.0/24、10.9.4.0/24）

② 続いて、staticd の設定

staticd の経路を、zebra がカーネルに入れる（10.9.20.0/24 は新規、10.9.1.0/24 は運用者と同じ）

③ FRR を止める

zebra が入れた 10.9.20.0/24 を消す。運用者の 4 本が残るかを見る

止める

実行例 1（2/3）: audit では、運用者の経路が消される

$ ip route show | grep ^10.9 （FRR 起動の 8 秒後）

```
10.9.1.0/24 via 10.1.0.2 dev d0 proto static metric 20
10.9.2.0/24 via 10.1.0.2 dev d0 metric 20
 (10.9.3.0/24 proto 196 と、10.9.4.0/24 proto 186 が無い)
10.9.20.0/24 nhid 13 via 10.1.0.3 dev d0 proto 196 metric 20
```

cpown の出力（抜粋、audit は止めずに記録だけする）

```
0.000 AUDIT ROUTE4 replace 10.9.3.0/24 proto=196 [violation] by zebra[403] cg=/frr claim=mgmt-routes
0.000 AUDIT ROUTE4 delete 10.9.3.0/24 proto=196 [violation] by zebra[403] cg=/frr claim=mgmt-routes
0.000 AUDIT ROUTE4 delete 10.9.4.0/24 proto=186 [violation] by zebra[403] cg=/frr claim=mgmt-routes
2.094 AUDIT ROUTE4 replace 10.9.20.0/24 proto=196 [violation] by zebra[403] cg=/frr claim=mgmt-routes
# AUDIT violation 8
```

消えたのは proto 196 と 186 の 2 本。消した本人は zebra[403]（cgroup /frr）と、記録に残る。

止める

実行例 1（3/3）: enforce では、4 本とも残る

$ ip route show | grep ^10.9 （FRR 起動の 8 秒後。10.9.20.0/24 は入らない）

```
10.9.1.0/24 via 10.1.0.2 dev d0 proto static metric 20
10.9.2.0/24 via 10.1.0.2 dev d0 metric 20
10.9.3.0/24 via 10.1.0.2 dev d0 proto 196 metric 20
10.9.4.0/24 via 10.1.0.2 dev d0 proto bgp metric 20
```

cpown の出力（抜粋）と、zebra のログ

```
0.000 DENIED ROUTE4 replace 10.9.3.0/24 proto=196 [violation] by zebra[403] cg=/frr claim=mgmt-routes
0.000 DENIED ROUTE4 delete 10.9.4.0/24 proto=186 [violation] by zebra[403] cg=/frr claim=mgmt-routes
2.102 DENIED ROUTE4 replace 10.9.20.0/24 proto=196 [violation] by zebra[403] cg=/frr claim=mgmt-routes
zebra.log: netlink_send_msg error: Operation not permitted
zebra.log: default(0:254):10.9.3.0/24: Route install failed
```

止められた。ただし 10.9.20.0/24（zebra の新規経路）も、置換の印（NLM_F_REPLACE）が付くので拒否される。